

Test Strategy Document for app.vwo.com (A/B Testing Platform)

Prepared By: Senior Software Tester (20+ Years of Experience)

Reviewed By: QA Lead (20+ Years of Experience)

Date: May 3, 2025

🎯 Objective

The objective of this test strategy is to ensure **app.vwo.com** (A/B Testing Platform) delivers a seamless, secure, and high-performing user experience. We will test the platform's **end-to-end functionality, usability, performance, and security** to meet both business and technical requirements, ensuring zero critical defects in any functional or security area.

📋 Scope

✅ In Scope

- **Customer Workflows:**
 - **Sign-up / Registration:** Validating the user journey for creating a new account.
 - **Login / Logout:** Verifying that users can log in securely and log out as intended.
 - **Dashboard:** Ensuring users can view and interact with the dashboard, access test results, and manage experiments.
 - **Create A/B Test:** Verifying that users can create and configure A/B tests, including selecting variations, goals, and targets.
- **Account Management:**
 - User profile updates
 - Password reset functionality
- **Integration with Third-party Services:**
 - Integration with analytics tools (e.g., Google Analytics, Mixpanel)
 - API for test management and reporting
 - Webhooks for notifications
- **Web and Mobile Compatibility:**
 - Testing the platform on both desktop and mobile browsers.
- **API Testing & Integration Data Accuracy:**
 - Ensure proper integration between the A/B testing tool and third-party services via secure and reliable APIs.

❌ Out of Scope

- Native mobile applications (Android/iOS)

- Server-side backend testing
 - Non-production environments or integrations unrelated to the platform’s core functionality
-

3 Focus Areas

Focus Area	Key Considerations
Functional Correctness	Test user journeys, A/B test creation, results accuracy, login/logout, dashboard interactions.
Data Integrity	Verifying accurate reporting of A/B test results, metrics consistency, and goal tracking.
UI & Navigation	Validate UI consistency, layout, mobile responsiveness, error handling, and intuitive navigation.
Performance	Test load, stress, and scalability (using tools like JMeter or k6).
Security	Check for vulnerabilities, secure login, data privacy, and third-party integrations.
Authentication Security	Validate login functionality, session handling, and role-based access control.
Compatibility	Test across multiple browsers (Chrome, Firefox, Safari, Edge), operating systems, and devices.
Usability	Evaluate accessibility and ease of use with real users (WCAG 2.1 compliance).

4 Approach

- **Testing Types:**
 - **Black Box Testing:** Test scenarios based on business requirements without looking at internal code.
 - **White Box Testing:** Review internal code, APIs, and data flows for security and performance issues.
 - **Automated Testing:** Utilize **Selenium**, **TestNG**, or **Cypress** for functional test automation.
 - **Exploratory Testing:** Perform manual tests to uncover unanticipated issues, especially for edge cases.
 - **Load & Stress Testing:** Use tools like **JMeter** or **k6** to simulate high traffic and verify the platform can scale.

- **Security Testing:** Conduct penetration tests, vulnerability scans using **OWASP ZAP** and **Burp Suite**.
 - **Cross-Browser Testing:** Use **BrowserStack** or **Sauce Labs** to validate functionality across Chrome, Firefox, Safari, and Edge.
 - **Usability Testing:** Conduct usability tests with real users (5–10) to ensure the platform is intuitive and accessible.
 - **Database Testing:** Validate data integrity using **SQL queries** and ensure proper data storage for user actions and test results.
 - **Log File Validation:** Monitor and review log files for errors, API calls, and system exceptions.
-

5 Deliverables

- **Functional Test Reports:** Documenting all test cases executed and the corresponding results.
 - **API Test Report:** Documenting all API tests, including response time, error handling, and status codes.
 - **Performance Results:** Analysis of system behavior under load, response times, and throughput.
 - **Security Report:** Detailing identified vulnerabilities, with risk analysis and mitigation strategies.
 - **Cross-Browser Compatibility Report:** Report outlining results for different browsers and devices.
 - **Test Coverage & Defect Reports:** Coverage analysis and a defect report (JIRA).
 - **Automation Suite:** Collection of automated test cases for regression testing.
 - **UAT Report:** Final report for User Acceptance Testing with feedback from business stakeholders.
-

6 Team & Schedule

Team Composition:

- 4 Testers: 2 manual testers, 2 automation engineers
- 1 Performance Engineer
- 1 Security Tester (part-time)
- 1 QA Lead
- 1 UAT Coordinator

Schedule:

Month Activity

May Functional and Security Testing, Usability Testing

June Load/Performance Testing

July Compatibility Testing, UAT

August Regression Testing & Final Release

UAT Schedule:

- 5 Business Stakeholders + 5 Beta Users will be involved in UAT.
-

7 Entry & Exit Criteria

Entry Criteria:

- All user stories and requirements are signed off by the Product Owner.
- Development environment is stable with deployed code to be tested.
- Test data and test environments are set up.

Exit Criteria:

- 100% test cases executed with no critical/high defects open.
 - Performance metrics within acceptable limits (avg. response time < 2 seconds at 1000 users).
 - No critical vulnerabilities found in security testing.
 - UAT sign-off received from business stakeholders.
 - Regression testing conducted after major changes and prior to final sign-off.
-

8 Metrics to Track

- **Test Case Execution %:** Percentage of test cases executed compared to planned.
 - **Defect Density:** Number of defects found per functionality area.
 - **Test Coverage:** % of requirements covered by tests.
 - **Automation Coverage %:** The percentage of test cases automated vs manual tests.
 - **Performance Metrics:** Average response time, throughput, and system stability under load.
 - **Security Vulnerabilities:** Number of vulnerabilities found and their severity.
-

9 Risk Management & Mitigation

Risk	Mitigation Strategy
Frequent UI Changes	Keep test scenarios flexible, perform frequent regression testing.
API Inconsistencies	Use contract testing to verify APIs independently of front-end.
Performance Bottlenecks	Identify early and resolve with optimization before the final release.
Limited UAT Participants	Pre-identify backup UAT testers and ensure business availability.
Integration Delays	Mock third-party services during early testing stages.
Unexpected Third-party Service Downtimes	Implement retries and mock services during testing to simulate outages.

10 Tools Stack

Type	Tools
Test Management	Jira, TestRail
Automation	Selenium, TestNG, Cypress
API Testing	Postman, Rest Assured
Performance Testing	JMeter, k6
Security Testing	OWASP ZAP, Burp Suite
Cross-Browser Testing	BrowserStack, Sauce Labs
Usability Testing	Axe, Lighthouse
CI/CD	Jenkins, GitHub Actions

Approval

Role	Name	Signature	Date
QA Manager	[Name]	(signed)	03-May-2025
QA Lead	[Name]	(signed)	03-May-2025
Product Owner	[Name]	(pending)	—